Mobile Demo Script Explanation -- Nojumpstarts (Exploitation, Challenge 02)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: The 'signed message chain' means nothing once the RSA private key itself is a compile-time constant in the bytecode.

Background -- what the terms below mean

What does "Exploitation" mean, as a category? An **Exploitation** challenge gives you a real target Android app with an actual security mistake in it, and asks you to write a second, small app of your own that takes advantage of that mistake to get the flag.

What is a vulnerability? A mistake in how software was built that lets someone do something the developer never intended -- here, get the flag without going through the app the normal, intended way.

What is an APK, and what does "decompile" mean? An APK is the installable file for an Android app. It ships as compiled code that isn't directly readable, so we **decompile** it -- turn it back into readable, Java-like source code -- using a free tool called **jadx** (opened as **jadx-gui**) -- to find the mistake in the first place.

What is a package name? Every Android app has a unique ID string (like org.naija.fdexploit). Android refuses to install two different apps that both use the exact same ID, which occasionally trips up a first submission attempt if the exploit app accidentally reuses the target's own package name.

How the APK was obtained

The target APK was **downloaded directly from the challenge's own page** on challs.reyammer.io, before doing anything else. A copy of it is kept at:

`C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/02 Nojumpstarts\Original Target APK.apk`

How the solution was built

Built as a normal **Android Studio project** (Gradle). From a terminal: `cd` into its `Android Studio Project` folder and run `gradlew.bat assembleDebug`. The resulting APK is kept right there as **SUBMITTED APK Challenge 02 Nojumpstarts.apk**, ready to re-upload.

The vulnerability / technique, in plain terms

Embedded RSA private key: Main.PRIVKEY holds the full RSA private key as a compile-time constant (read directly from the class_def's static_values table), completely defeating the MainActivity to A to B to C signed-message chain meant to prevent component hijacking. We forge a valid signature offline for C's expected final message and launch C directly, skipping A and B entirely. First submission logged nothing (harness only surfaces MOBISEC-tagged lines) -- fixed by adding full Log.e instrumentation.

Files to open

1. Open the target APK in jadx-gui:

`C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/02 Nojumpstarts\Original Target APK.apk`

2. MainActivity.java:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/02 Nojumpstarts\Android Studio Project\app\
```

This is our own code -- open directly in Android Studio or any text editor.

What to say -- explanation of the exploit, step by step

1. I decompiled the target and found Main.PRIVKEY holds the full RSA private key as a compile-time constant in the bytecode -- not just the public key.

2. I used that key to forge a valid signature for the exact message activity C expects, letting me launch C directly and skip the intended MainActivity to A to B to C chain entirely.

3. My first submission logged nothing, since the harness only surfaces MOBISEC-tagged lines; I fixed this by adding full Log.e instrumentation at every step.

4. I resubmitted and the platform's analysis log confirmed the flag was returned.

If an emulator isn't available

If asked how you got/confirmed the flag: "I built my exploit app and uploaded the APK to the platform's own submission page. Their server ran it automatically against the target and returned the flag in the analysis log." This is backed by a real screenshot of that exact submission page (see **Real Screenshot: Live Platform Submission Page** in this challenge's Report.pdf), not just a claim -- worth pointing to directly if an emulator isn't running.

Result

The flag this challenge produced:

```
MOBISEC{you_shall_not_jump_ok?pretty_please?}
```

Source: Verbatim from /apk/81742 analysis log

The real, verbatim output this run produced:

```
njsexploit: building intent for C
njsexploit: onActivityResult resultCode=-1 data=Intent { (has extras) }
njsexploit: FLAG: MOBISEC{you_shall_not_jump_ok?pretty_please?}
```